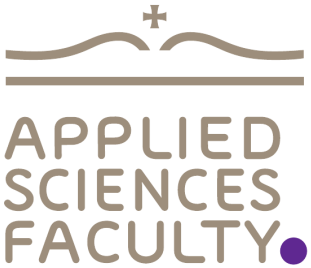
C++ – модулі, C++20+.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- Поділ програм на модулі за допомогою файлів заголовків має багато проблем – вони не раз обговорювалися протягом цього курсу.
- Модулі – концепція, добре відпрацьована в інших мовах.
- C++, як завжди, пішли своєю дорогою – надали максимально гнучкий, але складний у використанні засіб.
- Підтримка компіляторами станом на початок 2023 присутня, але в GCC ще не повна[1].
- Підтримка в CMake є, але експериментальна і потребує певного шаманства[2].
- Використовувати із Makefile можна, але потребує трохи заморочитися.
- Імена файлів модулів поки не стандартизовані. MSVC рекомендує .ixx для інтерфейсів модулів[3], Clang використовує .cppm, GCC не має вподобань, або автору не вдалося їх знайти.
- Поки використано розширення .cxx – GCC його розпізнає як код C++ без вказання мови, і не створює проблем іншим.

[1] `https://github.com/royjacobson/modules-report`.

[2] `https://www.kitware.com/import-cmake-c20-modules/`.

[3] Див. далі.

- Найпростіший модуль – в одному файлі. Всі сутності, позначені export можна використовувати у файлах, що імпортують:

Модуль

```
export module first_module;

export const char* get_hello() {
    return "Hello, world!";
}
export struct X_t{};
export int val = 5;

export template<typename T>
void fn(T){}

export template<typename T>
struct templ_t{};

struct Y_t{};
int local_val = 11;
```

Використання

```
import first_module;
import <iostream>;
import <iomanip>;
import <string>;
int main(){
    X_t x1;
    // Y_t y1; // Not exported --
        // cannot be accessed
    std::cout << get_hello()
              << std::endl;
    std::cout << val << std::endl;
    fn(1);
    templ_t<int> t1;
    templ_t<std::string> t2;
    return 0;
}
```

- Зверніть увагу – перевикористано ключове слово export.

Технічні подробиці для gcc

- Стандартні файли заголовків не представлені як importable header units[4] – перш ніж їх так використовувати, потрібно їх скомпілювати:

```
g++ -std=c++20 -fmodules-ts -x c++-system-header
        iostream iomanip string -c
```

- Компіляція:

```
g++ -c -std=c++20 -fmodules-ts -Og first_module.cxx \
                           -o obj/first_module.o
g++ -c -std=c++20 -fmodules-ts -Og l34_modules_demo_main.cpp \
                           -o obj/l34_modules_demo_main.o
g++ -std=c++20 -fmodules-ts -Og obj/l34_modules_demo_main.o \
              obj/first_module.o -o bin/l34_modules_demo
```

- За замовчуванням модулі будуть в директорії gcm.cache.
- ◇ Керувати цим можна за допомогою так званих mapper-ів, див. `-fmodule-mapper=mapper.txt`.
- ◇ Працює станом на 05-2024. На жаль, хоча базово модулі працюють і під MinGW, змусити працювати із стандартними заголовками станом на 05-2023 не вдалося – ймовірно, через різні способи трансляції шляхів Windows у різних частинах.

[4] Додано в C++23, станом на травень 2024 не підтримуються GCC чи Clang.

- Має проблеми із стандартними заголовками, якщо їх включати include.
- ◇ Так нормально:

```
import  hello ;

#include <iostream>
```

- ◇ Так будуть помилки компіляції, якщо модуль теж включає щось із стандартної бібліотеки:

```
#include <iostream>
import  hello ;
```

- ◇ Ймовірна причина проблеми – десь поміж відсутності реалізації "Partition definition visibility rules", "Textual merging of reachable GM entities" (головна версія) чи "Language-linkage module attachment".

CLang (DRAFT)

- Щось типу:

```
clang++ -Wall -std=c++2a -fimplicit-modules
     -fimplicit-module-maps -c -Xclang
     -emit-module-interface -fbuiltin-module-map
  first_module.cxx -o first_module.pcm
```

  однак, потребує ручного створення модулів стандартної бібліотеки.
- Детальніше, як це можна зробити, див. `https://stackoverflow.com/a/67323138` – відповідь на "Import std lib as modules with clang".
- Дивно, що колись воно працювало: `https://github.com/andrewvaughanj/cpp20_module_examples`.
- Має простіший за маппери GCC спосіб вказати, куди зберігати прекомпільовані модулі. Хоча свій маппер теж має.
- Має проблеми із стандартною бібліотекою (2022) – якщо не робити магії із stackoverflow, відмовляється знаходити навіть string.

MSVC

- Оскільки C++20 не описав модулі для стандартної бібліотеки, кожен виробник пішов своїм шляхом.
- MSVC використовує свій синтаксис:

```
import std.core;
```

- Також вона нав'язує розширення файлу модулів – це має бути .ixx.

```
cl /std:c++latest /EHsc /experimental:module /MD
                          /c first_module.ixx
cl /std:c++latest /EHsc /experimental:module /MD
                         /c l34_modules_demo_main_msvc.cpp
cl /std:c++latest /EHsc /experimental:module
         /MD first_module.obj l34_modules_demo_main_msvc.obj
         /Fe:l34_modules_demo_main_msvc
```

- Формат компільованого модуля GCC – Compiled Module Interfaces (CMI), із розширенням .gcm. Clang – Binary Module Interface (BMI), теж із .gcm. Для MSVC – IFC, розширення .ifc.

C++23 – модулі стандартної бібліотеки

- В C++23 додали `import std;` – "хедери" C++, та `import std.compat;` – C:

```
import std;
import std.compat;

int main() {
    std::println("Hello World!");
    printf("Hello World!\n");
    return 0;
}
```

- Відповідний документ: `wg21.link/p2465`.

Сутності, пов'язані із модулями

- Будь-який файл коду із module називається module unit (“блок модуля”?).
- Вони бувають декількох видів:
  - ◇ Блок-інтерфейс модуля (module interface unit) – модуль із ”export module”. Їх може бути декілька.
  - ◇ Блок-реалізація модуля (module implementation unit) – всі інші блоки модуля, без ”export module”.
  - ◇ Module-partition (”розділ модуля”?) – містить спеціальний синтаксис для розділів, див. далі. Не підтримується GCC15 і Clang15 (чи підтримується в Clang17 – не впевнений).
  - ◇ Module interface partition – розділ, що містить також інтерфейс.
  - ◇ Module implementation partition – розділ, що не містить інтерфейсу.
  - ◇ Головний блок інтерфейсу модуля (primary module interface unit) – не розділ, що містить інтерфейс модуля.
  - ◇◇ Має бути виключно один, решта блоки-інтерфейси модуля мають бути розділами.
- Приклад вище містив у собі всі частини зразу.

- Приклад базується на `https://vector-of-bool.github.io/2019/03/10/modules-1.html`.
- Монолітний модуль:

```
module;

#include <string>

export module hello;

export std::string get_hello_en() {
    return "Hello!";
}
export std::string get_hello_ua() {
    return "Привіт!";
}
```

- Все, що йде після "export module" належатиме до так званого purview модуля – його "активної частини". Подробиці – далі. Тому будь-які include там – погана ідея.
  - ◇ Однак, якщо почати модуль ключовим словом module – без експорту, то там можна розташувати **директиви препроцесора** (і лише їх!) – код звідти не стане частиною purview модуля.
- Нехай ми вирішили його розділити на частини.

Поділ модуля на частини – розділи (partition)

- Блок інтерфейсу модуля:

```
export module hello;

export import : english ;
export import : ukrainian ;
```

- Розділ інтерфейсу української мови:

```
module;
#include <string>

export module hello: english ;

export std :: string  get_hello_en() {     return "Hello!";     }
```

- Розділ інтерфейсу англійської мови:

```
    .....
export module hello: ukrainian ;

export std :: string  get_hello_ua() {     return "Привіт!";     }
```

- Використання не змінилося.
- Ремарка: модуль має експортувати всі свої інтерфейсні розділи.

Особливості розділів

- Станом на 2024, GCC15 і Clang15 не підтримують розділи, (чи підтримує Clang17 – під питанням).
- Модуль має експортувати всі свої інтерфейсні розділи.
- ◇ Але no diagnostic required – може бути UB.

Поділ модуля на частини – дрібніші модулі

- Блок інтерфейсу модуля – зверніть увагу на заміну ':' на '.'

```
export module hello;

export import hello.english;
export import hello.ukrainian;
```

- Модуль інтерфейсу української мови:

```
    ...................
export module hello.english;

export std::string get_hello_en() {
    return "Hello!";
}
```

- Модуль інтерфейсу англійської мови:

```
    ...................
export module hello.ukrainian;

export std::string get_hello_ua() {
    return "Привіт!";
}
```

- Використання не змінилося.
- Крапка в імені модуля – декоративна, не має спеціального значення.

Блок реалізації модуля

- Повернемося до початкового, монолітного, модуля, але винесемо реалізацію окремо.
- Інтерфейс:

```
module;
#include <string>

export module hello;

export std::string get_hello_en();
export std::string get_hello_ua();
```

- Реалізація:

```
module;
#include <string>

module hello;

std::string get_hello_en() { return "Hello!"; } // no export

std::string get_hello_ua() { return "Привіт!"; } // no export
```

- Використання не змінилося.

- Розділи теж можна розділяти на блок реалізації розділу і блок інтерфейсу розділу.
- "export import" можна застосовувати лише до інтерфейсних блоків модуля чи інтерфейсних розділів – він позбавлений сенсу для блоків реалізацій.
- "export module", яке визначає головний інтерфейсний блок модуля, повинен бути єдиним.
- Блоки реалізації неявно імпортують блоки інтерфейсу, тому бачать всі сутності, оголошені там. Зробити це явно – імпортувати блок реалізації, неможливо.
- Головний інтерфейсний блок бачить все, оголошене в інших блоках.
- Тіло шаблонів (все ще) має бути експортованим для успішної компіляції.
  - Можна спробувати виносити його в розділи, але, оскільки розділ не бачить інших частин (без явного імпорту), зокрема – головного блоку, це слабо допомагає...
- Теоретична перевага розділів над окремими модулями – краща сумісність ABI.

- Конструкція, використана на початку:

  import <iostream>;
  import <some−third−party.hpp>;
  import "some−local.hpp";

  називається "header-unit import"[5]
- Компілятор може згенерувати його із файлу заголовків, експортуючи все, що є експортабельного у "хедері".
- Однак, "транзієнтний" стан препроцесора не впливає на імпорт – іншими словами, жодні макроси, визначені тим, хто імпортує, не впливає на обробку хедера.
- Це дуже суперечить поширеним практикам!
- Глобальні макроси, задані, наприклад, із командного рядка компілятора, враховуються.
- Header-unit можна "export import", але, знову ж, макроси не експортуються.
- Які хедери є importable – на розсуд реалізації.
  - їй може бути потрібен, наприклад, наперед складений список. Як от із GCC.
  - Але всі стандартні C++ хедери мають бути імпортувабельними. Хедери C – не обов'язково.

[5] Колись називалося legacy header-unit import.

Global module

- Все, що не є частиною модулів – іншими словами, весь код до-модульної епохи, і те, що не є безпосередньою частиною модулів, вважається таким, що належить до глобального модуля чи модуля без імені.
- Зокрема, туди ж потрапляє все між `module;` і `module name;` чи `export module name;`.
  - ◇ Ця частина називається global module fragment.
- Module headers – спосіб імпортувати щось із модуля без імені (що неможливо зробити явно – через відсутність імені у нього).

private частина модуля

- Опціонально модуль може містити приватну частину:

```
module;
#include <string>
export module hello;

export std::string get_hello_en();
export std::string get_hello_ua();

module :private;

std::string get_hello_en() { return "Hello!"; }
std::string get_hello_ua() { return "Привіт!"; }
```

- Технічно це спосіб відділити приватну частину, що не має жодним чином впливати на інтерфейс від інтерфейсної, не розділяючи модуль на декілька блоків-файлів.

- Оскільки зміна private-частини не має потребувати перекомпіляції користувачів модуля, на неї є багато обмежень.
  - Наприклад, inline-функції були б проблемою.
- Для потреб підтримки такої ізоляції методи класів, визначені в оголошенні більше не вважаються неявно inline.
- Модуль із приватною частиною має складатися виключно з одного файлу.

Linkage – види зв'язування

- Internal linkage – не доступні за межами одиниці трансляції. Від `static` і `namespace{ ... }` до констант.
- External linkage – сутності, з якими можна лінкуватися з інших одиниць трансляції.
  - Включає `extern "C"` і `extern "C++"` – види експортування.
    - Формально, оці extern – language linking.
  - Завдяки ODR – мають бути однакові у всіх одиницях трансляції.
- Module linkage – доступні для лінкування в межах одного модуля, але не за його межами. Це, грубо кажучи, все не експортоване з модуля.
  - Теж потрібно дотримуватися аналогу ODR в межах одного модуля.
  - Все, що йде після export module буде мати module linkage (див. далі)
- No linkage – без зв'язування, локальні змінні тощо.
- Див. також `https://www.cppstories.com/2023/storage-init-cpp/`.

Видимість

- Сутність із модуля, яку можна використати безпосередньо – експортовану, називають видимою (visible).
- Є також поняття доступності (reachable)– її можна використовувати за межами модуля, але – не безпосередньо.
- Приклад. Нехай вміст модуля такий:

```
export module stuff;

class C {                    .................
public:
        void do_smth();
};
extern C get_C() {}
```

- Тоді клас C не є видимим, але є доступним, а функція get_C() – видима і доступна:

```
// C c; // Error!
auto c = get_C();
c.do_smth();
```

Можна (але не треба) навіть так:

```
using stolen_C = decltype(c);
stolen_C sc;
```

Що можна експортувати

- Не можна експортувати сутності із внутрішнім зв'язуванням. Зокрема – частини анонімних просторів імен, статичні змінні та функції, константи.
- Можна еспортувати лише сутності на рівні просторів імен, включаючи глобальний:

```
export struct C { // OK
    int a;
    int b;
};

struct D { // Error!
    export int a;
    int b;
};
```

- Експортуються using-оголошення, якщо вихідну сутність можна експортувати, але не using namespace:

```
export using some_namespace::some_class;
// export using namespace some_namespace; // Error
```

- Можна експортувати цілий простір імен, але в ньому все має бути експорутувабельним:

  export namespace some_namespace { ..... }

  Це обмеження не стосується наступних розширень простору імен:

  export namespace some_namespace { ..... }

  ```
  namespace some_namespace {
    .....
          static int x = 0;
  }
  ```

- Можна скористатися export-блоком:

  ```
  export {
          class C{};
          void fn();
  }
  ```

- Оголошень може бути декілька, але якщо сутність експортується, перше із них має використати ключове слово export. Далі можна і без нього.
- Якщо щось експортується з простору імен, він сам теж експортується (неявно), але інші його сутності, самі по собі – ні, їх треба експортувати явно.

Що можна імпортувати

- import – контекстозалежне ключове слово.
  - ◇ Але не називайте так ідентифікатори!
- Всі імпортування мають бути до перших оголошень.
- Імпортувати можна лише в глобальній області видимості.
- Себе чи чужі розділи модуль не може імпортувати.
- Модуль реалізації неявно імпортує свій інтерфейсний модуль – однак, із реалізацією шаблонів це не допомагає, оскільки, наприклад, всі реалізації методів, винесені з оголошення, потрібно експортувати теж.

MSVC – модулі стандартної бібліотеки

- std.core – основна частина.
- std.regex – <regex>
- std.filesystem – <filesystem>
- std.memory – <memory>
- std.threading – <atomic>, <condition_variable>, <future>, <mutex>, <shared_mutex>, <thread>.
- C++23 стандартизував інший підхід: модуль std для стандартної бібліотеки C++ і std.compat для C.

Посилання

- Дуже грунтовний огляд модулів:
  - `https://vector-of-bool.github.io/2019/03/10/modules-1.html` – "Understanding C++ Modules: Part 1: Hello Modules, and Module Units",
  - `https://vector-of-bool.github.io/2019/03/31/modules-2.html` – "Understanding C++ Modules: Part 2: export, import, visible, and reachable",
  - `https://vector-of-bool.github.io/2019/10/07/modules-3.html` – "Understanding C++ Modules: Part 3: Linkage and Fragments".
- `https://vector-of-bool.github.io/2019/01/27/modules-doa.html` ("C++ Modules might be Dead-on-Arrival") та `https://vector-of-bool.github.io/2019/03/04/modules-doa-2.html` ("Are C++ Modules Dead-on-Arrival?") – про проблеми модулів.

- https://docs.microsoft.com/en-us/cpp/cpp/tutorial-named-modules-cpp?view=msvc-170 – модулі в MSVC 2022.
  - Див. також https://devblogs.microsoft.com/cppblog/using-cpp-modules-in-msvc-from-the-command-line-part-1/.
- https://clang.llvm.org/docs/Modules.html – модулі в Clang.
- https://blog.feabhas.com/2021/08/c20-modules-with-gcc11 – "C++20 modules with GCC11"
- https://gcc.gnu.org/onlinedocs/gcc-12.1.0/gcc.pdf – глава "3.23 C++ Modules".
- https://en.cppreference.com/w/cpp/language/modules
- https://arewemodulesyet.org/ – дещо іронічний сайт, що відстежує підтримку модулів.
- "Can we finally use C++ Modules in 2026?" – поки ще не дуже...
  - "C++20 Modules Don't Spark Joy" (14 квітня 2026) – аналогічний висновок.
- "C++20 Modules: Best Practices from a User's Perspective", 30 грудня 2025.
- "C++20 modules and Boost: a prototype", січень 2025 – поки вирішили відкласти.

